ASSIGNMENT_CO1

Jithesh C(192524320)

Github link: https://github.com/jithesh192524320/DSA0120-OBJECT-ORIETED-PROGRAMMING-WITH-C-.git

TASK 1 – Understand the Problem and Identify the Objects

1. Object-Oriented Analysis

The Namma Metro Smart-Card Fare System can be divided into the following objects:

1. SmartCard: Stores the card ID and balance. It checks whether the passenger has enough balance, deducts the fare and supports recharge.
2. Passenger: Stores passenger ID, name and category such as regular, student or senior citizen.
3. Journey: Stores the origin zone, destination zone, travel hour and calculated fare.
4. Zone: Represents the Inner, Middle and Outer zones. Each zone has a different base fare.
5. FareTable: Stores the base fares and surcharge percentage. It calculates the fare according to the travel zone and peak hours.

2. Procedural Programming vs Object-Oriented Programming

In procedural programming, data such as balance, fare and passenger category are stored in separate variables. Functions such as calculateFare() and deductFare() perform the operations.

In object-oriented programming, data and functions are grouped into classes such as SmartCard, Passenger, Journey and FareTable.

OOP improves code organisation, reusability and data protection. It also makes the program easier to maintain.

3. Four Principles of OOP

- Encapsulation: Keeps the smart-card balance private and allows changes only through class methods.

- Abstraction: Hides internal fare calculation details and provides a simple function to calculate the fare.
- Inheritance: Student and senior passenger classes can inherit common details from a Passenger class.
- Polymorphism: Different passenger categories can have different concession rates using a common virtual function.

4. Rules of the System

- Inner Zone fare – Rs. 10
- Middle Zone fare – Rs. 20
- Outer Zone fare – Rs. 30
- Peak-hour surcharge – 20%
- Peak hours – 8:00 AM to 10:00 AM and 5:00 PM to 8:00 PM, excluding 10:00 AM and 8:00 PM.
- Minimum remaining balance after a journey – Rs. 50.
- The balance must never become negative.
- Money should be stored as integer paise to avoid floating-point rounding errors.

The assignment does not specify exact student and senior citizen concession rates. These rates must be confirmed before implementing concessions.

5. Code:

```
#include <iostream>

#include <string>

using namespace std;


class SmartCard {

private:
```

```
    int balance;

public:
    SmartCard(int b) {
        balance = b;
    }

    void displayBalance() {
        cout << "Card Balance: Rs " << balance << '\n';
    }
};

class Passenger {
public:
    string name;
    int id;

    Passenger(string n, int i) {
        name = n;
        id = i;
    }

    void display() {
        cout << "Passenger: " << name << '\n';
```

```
        cout << "Passenger ID: " << id << '\n';
    }
};

class Journey {
public:
    string source;
    string destination;

    Journey(string s, string d) {
        source = s;
        destination = d;
    }

    void display() {
        cout << "Source: " << source << '\n';
        cout << "Destination: " << destination << '\n';
    }
};

int main() {
    Passenger p("Arun", 101);
    SmartCard card(100);
    Journey j("Inner", "Outer");
```

```
    p.display();

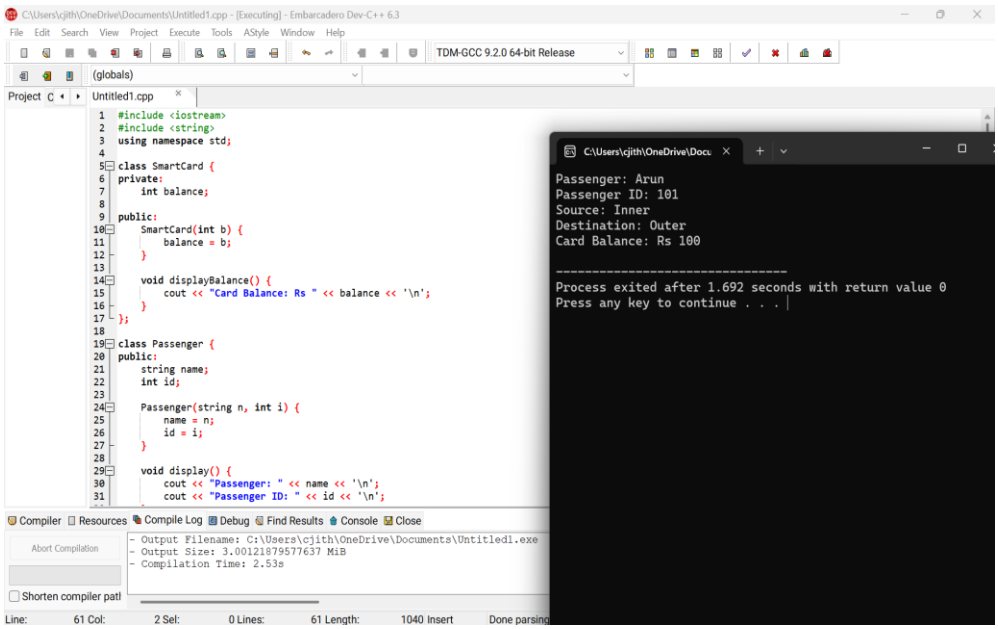
    j.display();

    card.displayBalance();


    return 0;

}
```

6. Output:



7. Complexity Analysis

Both procedural and object-oriented approaches take O(1) time for each fare calculation and use O(1) memory for each card.

OOP makes the system easier to maintain because fare rules and card operations are organised into separate classes.

TASK 2 – Tokens, Keywords, Identifiers and Constants

1. Five Types of C++ Tokens

1. Keywords: Reserved words in C++, such as int, class, if, return and switch.
2. Identifiers: Names created by the programmer, such as SmartCard, balance and calculateFare.
3. Constants/Literals: Fixed values such as 5000, 20, 10.0 and true.
4. Operators: Symbols used to perform operations, such as +, -, *, /, % and ==.
5. Punctuators: Symbols used to structure a program, such as ;, ,, (), {} and [].

2. Constants and Zone Declaration

```
#include <iostream>

using namespace std;

constexpr long long INNER_FARE = 1000;

constexpr long long MIDDLE_FARE = 2000;

constexpr long long OUTER_FARE = 3000;

constexpr int PEAK_PERCENT = 20;

constexpr long long MIN_BALANCE = 5000;


enum class Zone {

   Inner,

   Middle,

   Outer

};

int main() {

   cout << OUTER_FARE << '\n';
```

return 0;}

3. Difference Between #define, const and constexpr

- #define is a preprocessor directive that replaces text before compilation. It does not provide normal type checking.
- const creates a read-only variable whose value cannot be changed after initialisation.
- constexpr is used for values that can be evaluated at compile time.

constexpr is preferred for fixed fare values because it provides type checking and can be used in compile-time calculations.

4. Enum Class

enum class is safer than an ordinary enum because its values are scoped. For example, Zone::Inner clearly identifies the zone and does not automatically convert to an integer.

5. Code:

```
#include <iostream>

using namespace std;


constexpr int INNER_FARE = 1000;

constexpr int MIDDLE_FARE = 2000;

constexpr int OUTER_FARE = 3000;

constexpr int PEAK_SURCHARGE = 20;

constexpr int MIN_BALANCE = 5000;


enum class Zone {

    Inner,

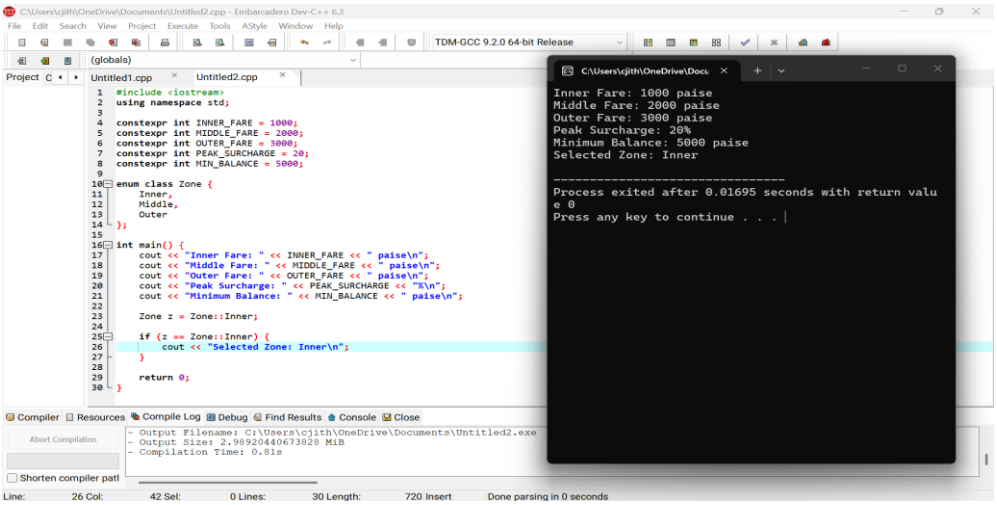
    Middle,

    Outer
```

```
};


int main() {
    cout << "Inner Fare: " << INNER_FARE << " paise\n";
    cout << "Middle Fare: " << MIDDLE_FARE << " paise\n";
    cout << "Outer Fare: " << OUTER_FARE << " paise\n";
    cout << "Peak Surcharge: " << PEAK_SURCHARGE << "%\n";
    cout << "Minimum Balance: " << MIN_BALANCE << " paise\n";
    Zone z = Zone::Inner;
    if (z == Zone::Inner) {
        cout << "Selected Zone: Inner\n";
    }
    return 0;
}
```

6. Output:



7. Complexity Analysis

Using constexpr and enum class improves type safety and readability. It does not change the O(1) time complexity of fare calculation.

TASK 3 – Choose Data Types and Check Type Compatibility

1. Suggested Data Types

Data	Suggested data type	Reason
Fare and balance	long long	Stores large money values in paise
Peak-hour status	bool	Stores true or false
Zone	enum class	Represents valid zone names
Travel hour	int	Stores values from 0 to 23
Journey count	long long	Supports large journey counts
Passenger initial	char	Stores a single character
Average and percentage	double	Stores decimal values

2. Why Integer Paise Is Better Than Double

For example, Rs. 10.20 can be stored as 1020 paise.

Integer calculations provide exact addition and subtraction for valid values. A double stores many decimal fractions approximately in binary, which can cause small rounding errors.

Therefore, long long is recommended for storing money in paise. Rupees can be displayed by dividing the paise value by 100.

3. Type Compatibility

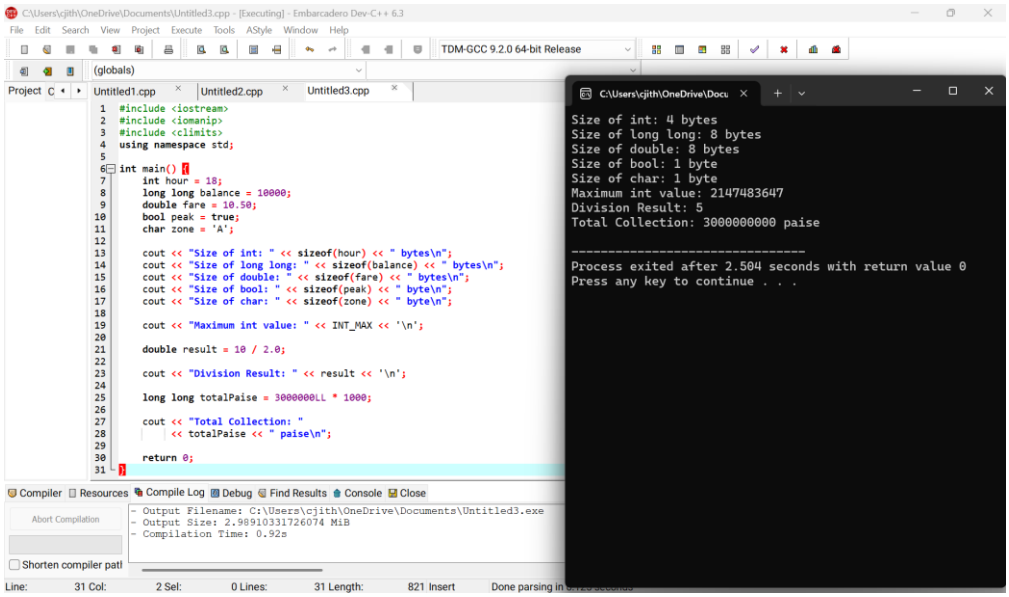
When different data types are used in one expression, C++ performs implicit type conversions.

- When int and double are added, the integer is converted into a double.
- When bool is used in arithmetic, it is converted to 0 or 1.
- When a decimal value is assigned to an integer, the fractional part is discarded.

4. C++ Program

```
#include <iostream>
#include <iomanip>
#include <climits>
using namespace std;
int main() {
    cout << "int bytes: " << sizeof(int)
         << " max: " << INT_MAX << '\n';
    cout << "long long bytes: " << sizeof(long long)
         << " max: " << LLONG_MAX << '\n';
    double total = 0.0;
    for (int i = 0; i < 1000000; ++i) {
        total += 0.1;
    }
    long long exactPaise = 0;
    for (int i = 0; i < 1000000; ++i) {
        exactPaise += 10;
    }
    cout << setprecision(17) << total << '\n';
    cout << exactPaise << " paise\n";
    return 0;
}
```

5. Output:



6. Network Collection Calculation

Assume 3,000,000 journeys, with each journey costing Rs. 30.

Total collection:

3,000,000 × 30 = Rs. 90,000,000

In paise:

90,000,000 × 100 = 9,000,000,000 paise.

This value exceeds the usual 32-bit signed integer limit. Therefore, long long is suitable for storing the total collection.

7. Complexity Analysis

Addition takes O(1) time for int, long long and double. Integer paise is preferred because it avoids decimal rounding problems in money calculations.

TASK 4 – Control Implicit Conversions

1. Explanation of Type Conversions

Integer division: When two integers are divided, the fractional part is removed. For example, 7 / 2 gives 3.

Explicit conversion: Using static_cast<double>(a) / b converts the integer into a double before division. The result is 3.5.

Character promotion: When a character is used in arithmetic, it is promoted to an integer. For example, 'A' + 2 produces 'C'.

Boolean conversion: Assigning a non-zero integer to a Boolean variable produces true. By default, true is printed as 1. Using boolalpha prints true.

Signed and unsigned comparison: Comparing a negative signed integer with an unsigned integer can convert the signed value into a large unsigned value.

Narrowing conversion: int x{3.7}; is rejected because brace initialisation prevents this narrowing conversion.

2. C++ Program

```
#include <iostream>

using namespace std;

int main() {

    int a = 7, b = 2;


    double c = a / b;

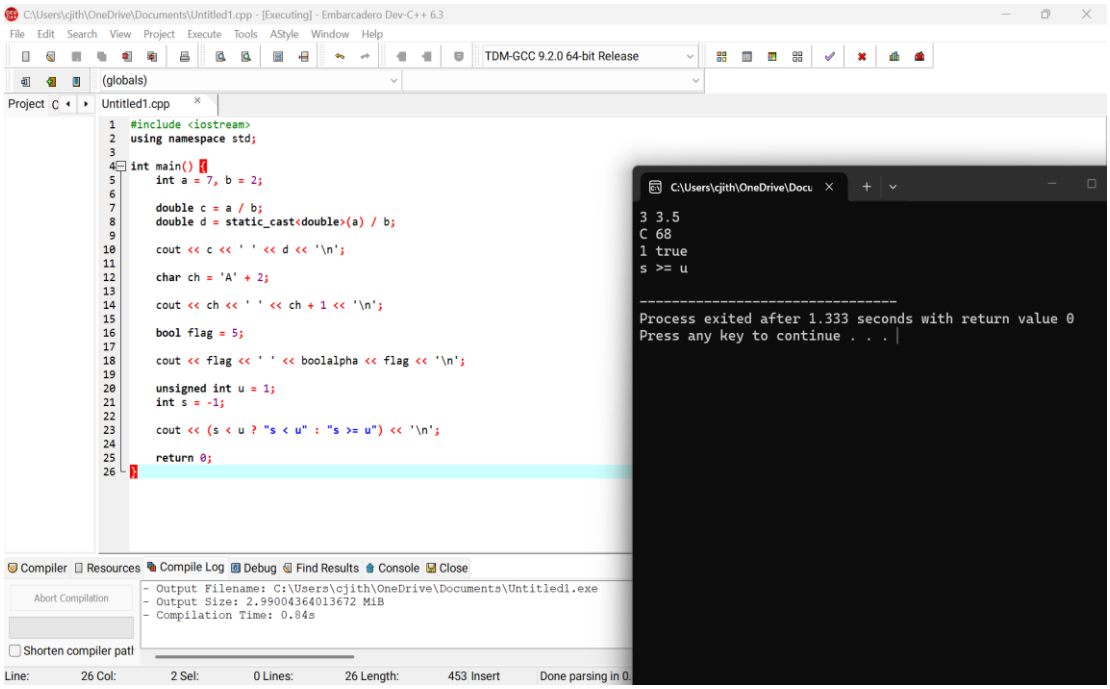
    double d = static_cast<double>(a) / b;


    char ch = 'A' + 2;
```

```
    bool flag = 5;
    unsigned int u = 1;
    int s = -1;
    cout << c << " " << d << '\n';
    cout << ch << " " << static_cast<int>(ch) + 1 << '\n';
    cout << flag << " ";
    cout << boolalpha << flag << '\n';
    if (s < u)
        cout << "s < u";
    else
        cout << "s >= u";

    return 0;
}
```

3. Expected Output



4. Explanation of Output

- 7 / 2 gives 3 because integer division removes the fractional part.
- static_cast<double>(a) / b gives 3.5.
- 'A' + 2 produces the character C.
- The character C has the integer code 67, so adding 1 gives 68.
- The integer 5 converts to Boolean true.

6. Complexity Analysis

Explicit conversions improve readability and make the programmer's intention clear. They do not normally change the runtime complexity or memory usage.

TASK 5 – Write Correct Expressions with the Right Precedence

1. Fare Calculation Expressions

bool peak = (hour >= 8 && hour < 10) ||

```
        (hour >= 17 && hour < 20);
long long farePaise = baseFarePaise;
if (peak) {
   farePaise += farePaise * 20 / 100;
}
farePaise -= farePaise * concessionPercent / 100;
```

2. Operator Precedence Explanation

- The && operator is evaluated before the || operator.
- Multiplication and division have higher precedence than addition and subtraction.
- Parentheses are used to make expressions clear.
- x++ returns the old value and then increments it.
- ++x increments the value first and then returns the new value.
- 10 / 3 gives 3 because integer division removes the fractional part.
- In C++17, -10 / 3 gives -3 and -10 % 3 gives -1.

3. Explanation of Output

- 5 + 3 * 2 gives 11 because multiplication is performed first.
- (5 + 3) * 2 gives 16 because parentheses are evaluated first.
- Post-increment uses the old value before increasing it.
- Pre-increment increases the value before using it.
- The conditional operator selects a value based on a condition.
- Integer division removes the fractional part.
- The final loop adds 0, 3, 6 and 9, producing 18.

4. Alternative Lookup-Table Approach

```
#include <iostream>
using namespace std;

int main() {
    bool peak[24] = {};

    for (int h = 8; h < 10; ++h) {
        peak[h] = true;
    }

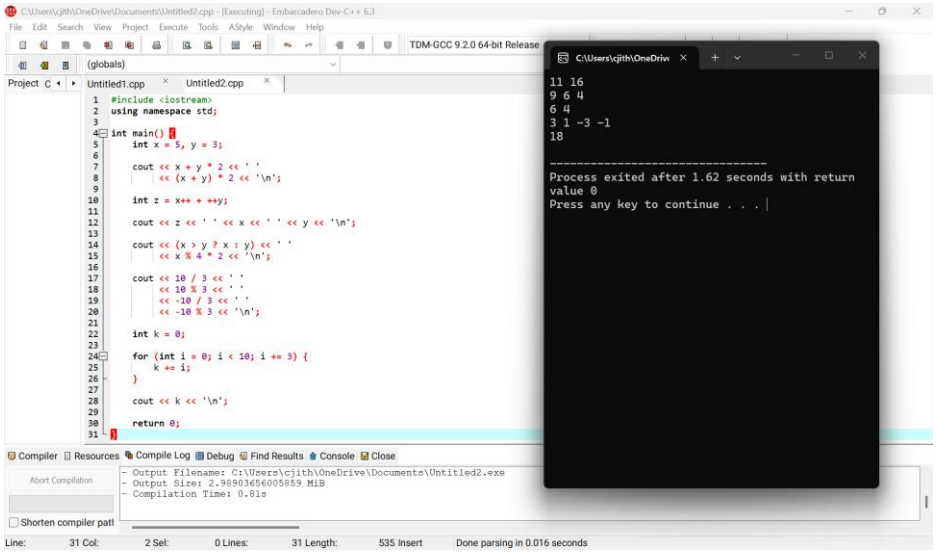
    for (int h = 17; h < 20; ++h) {
        peak[h] = true;
    }

    int hour = 18;

    if (hour >= 0 && hour < 24) {
        bool isPeak = peak[hour];
        cout << boolalpha << isPeak;
    }

    return 0;
}
```

5. Expected Output



6. Complexity Analysis

Both the direct expression and lookup-table approach take O(1) time. The lookup table uses additional memory for 24 Boolean values

TASK 6 – Design the Control Structures

1. Algorithm / Pseudocode

1. Start the program.
2. Read the zone, travel hour and card balance.
3. Check whether the zone and hour are valid.
4. Use a switch statement to select the base fare.
5. Check whether the journey takes place during peak hours.
6. If it is peak time, add a 20% surcharge.
7. Check whether the balance remaining after fare deduction is at least Rs. 50.
8. If the condition is satisfied, deduct the fare and display the remaining balance.
9. Otherwise, display journey denied.
10. Stop the program.

2. Completed C++ Program

```
#include <iomanip>
#include <iostream>
using namespace std;

enum class Zone {
    Inner,
    Middle,
    Outer
};

constexpr int BASE_PAISE[] = {
    1000, 2000, 3000
};

constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak) {
    long long base = 0;

    switch (z) {
        case Zone::Inner:
            base = BASE_PAISE[0];
            break;
```

```
        case Zone::Middle:
            base = BASE_PAISE[1];
            break;

        case Zone::Outer:
            base = BASE_PAISE[2];
            break;

        default:
            return -1;
    }

    if (peak) {
        base += base * PEAK_SURCHARGE_PERCENT / 100;
    }

    return base;
}

bool canTravel(long long balance, long long f) {
    return f >= 0 &&
           balance - f >= MIN_BALANCE_PAISE;
}
```

```
int main() {
    long long balance = 10000;

    struct Trip {
        Zone zone;
        bool peak;
    };

    Trip trips[] = {
        {Zone::Inner, false},
        {Zone::Outer, true},
        {Zone::Middle, false}
    };

    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);

        if (canTravel(balance, f)) {
            balance -= f;

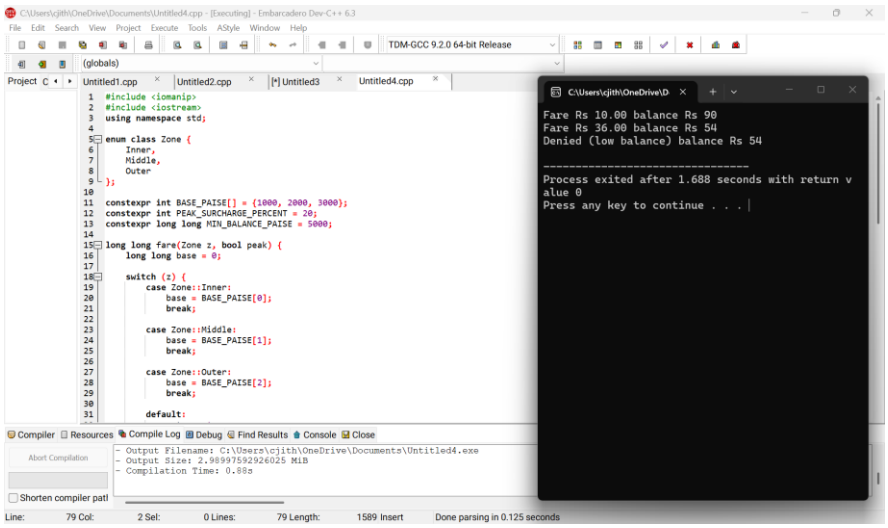
            cout << "Fare Rs "
                 << f / 100 << '.'
                 << setw(2) << setfill('0')
```

```
            << f % 100 << setfill(' ');
        } else {
            cout << "Denied (low balance)";
        }

        cout << " balance Rs " << balance / 100 << '\n';
    }

    return 0;
}
```

3. Output



4. Explanation of Control Structures

- if-else: Used to check conditions such as valid hours and sufficient balance.
- switch: Used to select the base fare according to the travel zone.
- for loop: Used to process multiple journeys.
- break: Used to exit a switch case or loop.

- continue: Used to skip an invalid journey and continue with the next one.
- Conditional operator: Used for simple condition-based value selection.

5. Complexity Analysis

Switch-based fare selection and array lookup take O(1) time. Processing multiple journeys takes O(n) time for n journeys.

TASK 7 – From Procedural to Object-Oriented

1. Procedural Programming

In procedural programming, data and functions are maintained separately. Functions operate on the balance passed to them.

2. Object-Oriented Programming

In object-oriented programming, data and functions are combined inside a class. The balance is declared private so that it cannot be directly modified outside the class.

3. Object-Oriented C++ Program

```
#include <iostream>

using namespace std;

class SmartCard {

private:

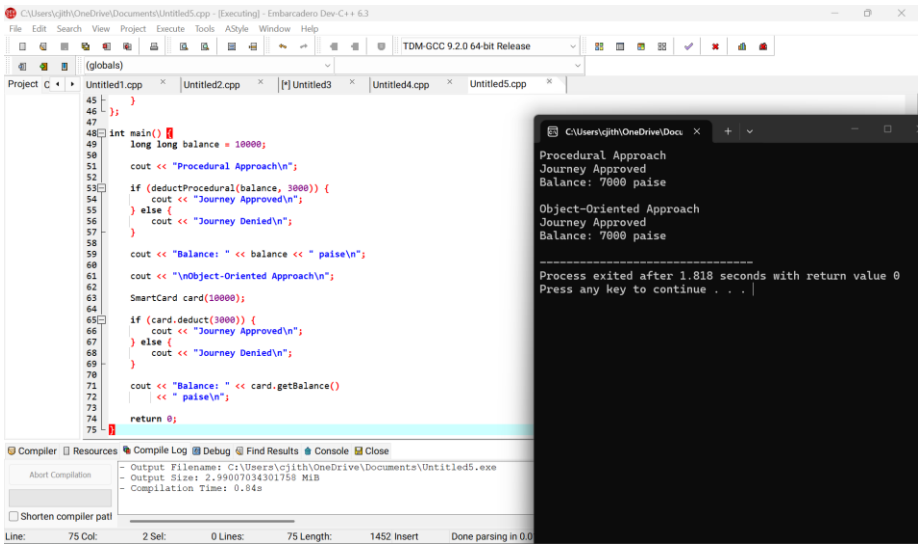
    long long balancePaise;

    static constexpr long long MIN_BALANCE = 5000;
```

```
public:
    explicit SmartCard(long long initial) {
        balancePaise = initial;
    }
    long long getBalance() const {
        return balancePaise;
    }
    bool canTravel(long long farePaise) const {
        return farePaise >= 0 &&
               balancePaise - farePaise >= MIN_BALANCE;
    }
    bool deduct(long long farePaise) {
        if (!canTravel(farePaise)) {
            return false;
        }
        balancePaise -= farePaise;
        return true;
    }
};
int main() {
    SmartCard card(10000);
    if (card.deduct(3000)) {
        cout << "Approved. Balance: "
             << card.getBalance() << '\n';
```

```
    } else {
        cout << "Denied. Balance: "
             << card.getBalance() << '\n';
    }
    return 0;
}
```

4. Output:



5. Difference Between Procedural and OOP

Procedural Programming	**Object-Oriented Programming**
Uses functions to perform operations	Uses classes and objects
Data is maintained separately	Data and functions are grouped

Data protection is limited	Encapsulation protects data
Reusability is less organised	Supports inheritance and polymorphism
Suitable for smaller programs	Suitable for larger applications

6. Complexity Analysis

Both programs take O(1) time for fare deduction and use O(1) memory for one card.

The OOP approach is safer because the balance is private and can only be changed through class methods.

TASK 8 – Read and Print Data Safely

1. Explanation

C++ uses cin to read input and cout to display output.

If the user enters letters instead of a number, cin enters a failure state. The following functions help handle invalid input:

- cin.fail() checks whether an input operation failed.
- cin.clear() resets the error state.
- cin.ignore() removes invalid characters from the input buffer.
- setw() sets the output field width.
- setfill() sets the padding character.
- fixed and setprecision(2) display two decimal places.

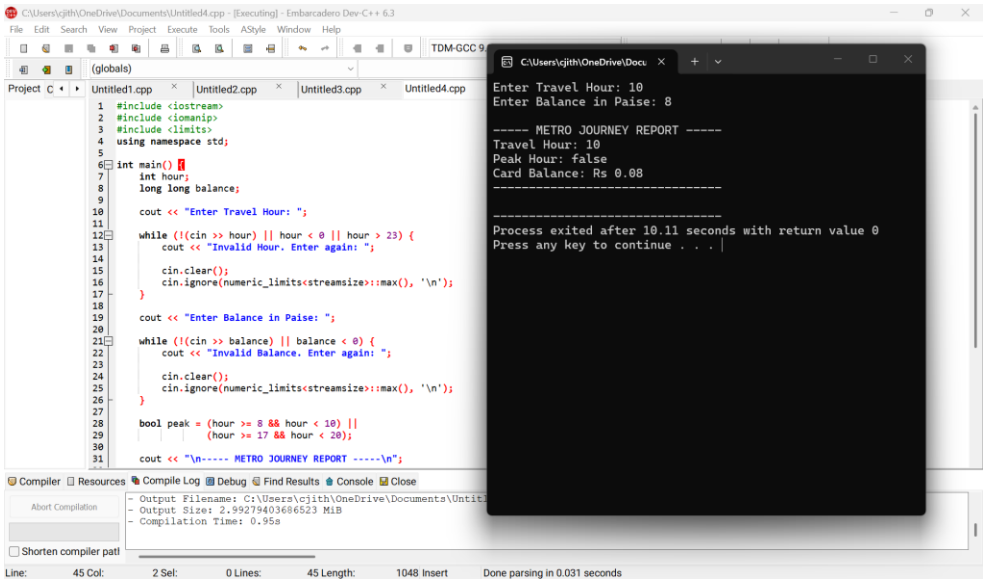
2. C++ Program

#include <iostream>

```
#include <iomanip>
#include <limits>
using namespace std;
int main() {
    int hour;
    cout << "Enter hour (0-23): ";
    while (!(cin >> hour) || hour < 0 || hour > 23) {
        cout << "Invalid hour. Enter an integer from 0 to 23: ";
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        }
    }
    long long balancePaise;
    cout << "Enter card balance in paise: ";
    while (!(cin >> balancePaise) || balancePaise < 0) {
        cout << "Invalid balance. Enter a non-negative amount: ";
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        }
    }
    bool peak = (hour >= 8 && hour < 10) ||
                (hour >= 17 && hour < 20);
```

```
    cout << boolalpha;
    cout << "Peak hour: " << peak << '\n';
    cout << fixed << setprecision(2);
    cout << "Balance Rs " << balancePaise / 100.0 << '\n';
    return 0;
}
```

3. Sample Output



4. Alternative Method

The getline() function can read a complete line. The stoi() function can convert a string into an integer. A try-catch block can handle invalid conversions.

5. Complexity Analysis

Reading and validating an input value takes O(1) time for a fixed-size numeric value.
Reading a full line and converting it takes O(L) time, where L is the line length.

TASK 9 – Implement and Test the Fare Logic

1. Fare Calculation Rules

Zone	**Base Fare**
Inner	Rs. 10
Middle	Rs. 20
Outer	Rs. 30

Peak-hour surcharge is 20%.

Peak hours:

- 8:00 AM to before 10:00 AM
- 5:00 PM to before 8:00 PM

The remaining balance after fare deduction must be at least Rs. 50.

2. C++ Program

```
#include <iostream>
#include <iomanip>
using namespace std;
enum class Zone {
    Inner,
    Middle,
    Outer
};
constexpr long long MIN_BALANCE = 5000;
```

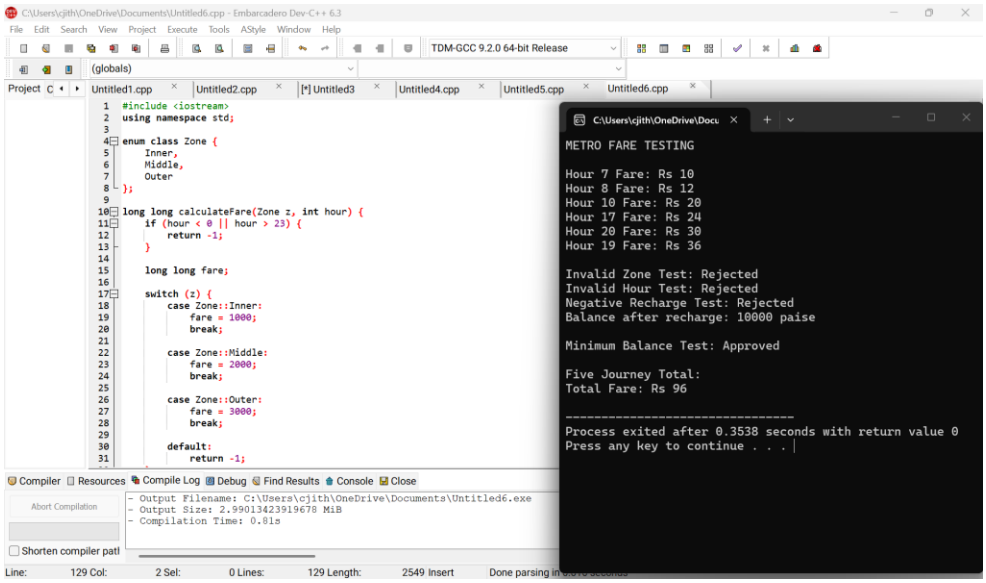
```
bool peakHour(int hour) {
    return (hour >= 8 && hour < 10) ||
           (hour >= 17 && hour < 20);
}
long long baseFare(Zone z) {
    switch (z) {
        case Zone::Inner:
            return 1000;


        case Zone::Middle:
            return 2000;
        case Zone::Outer:
            return 3000;
    }
    return -1;
}
long long calculateFare(Zone z, int hour) {
    if (hour < 0 || hour > 23) {
        return -1;
    }
    long long f = baseFare(z);
    if (f < 0) {
        return -1;
    }
```

```
    if (peakHour(hour)) {
        f += f * 20 / 100;
    }
    return f;
}
bool canTravel(long long balance, long long f) {
    return f >= 0 &&
           balance >= f &&
           balance - f >= MIN_BALANCE;
}
void printMoney(long long p) {
    cout << "Rs " << p / 100 << '.'
         << setw(2) << setfill('0')
         << p % 100 << setfill(' ');
}
int main() {
    struct Test {
        Zone zone;
        int hour;
        long long balance;
    };
    Test tests[] = {
        {Zone::Inner, 7, 10000},
        {Zone::Inner, 8, 10000},
```

```
        {Zone::Middle, 10, 10000},
        {Zone::Middle, 17, 10000},
        {Zone::Outer, 20, 10000},
        {Zone::Outer, 19, 9000}
    };
    for (const auto& t : tests) {
        long long f = calculateFare(t.zone, t.hour);
        cout << "Hour " << t.hour << ": ";
        if (canTravel(t.balance, f)) {
            printMoney(f);
            cout << " approved; remaining ";
            printMoney(t.balance - f);
        } else {
            cout << "denied or invalid";
        }
        cout << '\n';
    }
    return 0;
}
```

3. Output



4. Important Test Cases

- Inner zone at 7:00 AM – Rs. 10.
- Inner zone at 8:00 AM – Rs. 12.
- Middle zone at 10:00 AM – Rs. 20.
- Middle zone at 5:00 PM – Rs. 24.
- Outer zone at 8:00 PM – Rs. 30.
- Outer zone at 7:00 PM – Rs. 36.

5. Complexity Analysis

For t test cases, the program takes O(t) time. Each individual fare calculation takes O(1) time.

TASK 10 – Compare Expectations with Execution (4 Marks)

1. Calculation of Total Collection

Assume each journey costs Rs. 10.

Number of journeys = 3,000,000

Total collection:

3,000,000 × 10 = Rs. 30,000,000

Total in paise:

30,000,000 × 100 = 3,000,000,000 paise.

The usual maximum value of a 32-bit signed integer is 2,147,483,647. Therefore, this collection cannot safely be stored in a normal signed int.

long long is suitable because it supports a much larger range.

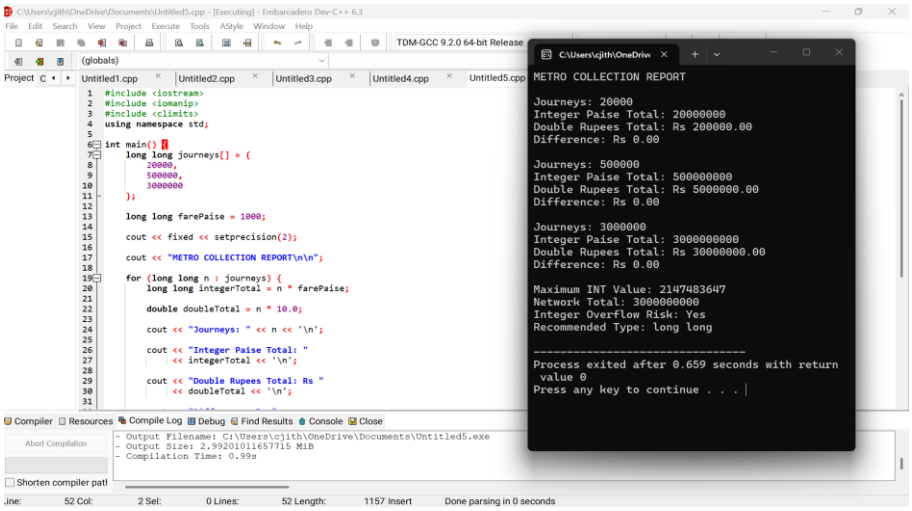
2. C++ Program to Compare Data Types

```
#include <iostream>
#include <iomanip>
#include <climits>
using namespace std;
int main() {
    const long long journeys = 3000000;
    const long long farePaise = 1000;
    long long exactTotal = journeys * farePaise;
    double doubleTotal = journeys * 10.0;
    cout << "INT_MAX = " << INT_MAX << '\n';
    cout << "Integer total paise = "
        << exactTotal << '\n';
    cout << fixed << setprecision(2);
    cout << "Double total rupees = "
        << doubleTotal << '\n';
    cout << "long long bytes = "
        << sizeof(long long)
        << ", double bytes = "
        << sizeof(double) << '\n';
```

```
    return 0;

}
```

3. Output



4. Expected Result

- int may overflow when the total collection exceeds its maximum range.
- long long can store the large total in paise.
- double can store large values, but decimal calculations may have rounding errors.
- Integer paise is the recommended method for fare calculations.

5. Complexity Analysis

Each addition takes O(1) time. The memory used by each variable is constant. The actual compiler output and any warnings should be recorded after compiling the program.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

1. Final Design Recommendation

The following design is recommended for the Namma Metro Smart-Card System:

1. Use enum class Zone to represent the Inner, Middle and Outer zones.
2. Store fare and card balance using long long in paise.

3. Use constexpr for fixed fare values and surcharge percentages.
4. Use a SmartCard class with a private balance.
5. Use a central FareTable or fare calculation function.
6. Validate the travel zone and hour before calculating the fare.
7. Apply the 20% surcharge only during peak hours.
8. Maintain a minimum remaining balance of Rs. 50.
9. Use long long for the total collection of the entire metro network.

2. Advantages of the Recommended Design

- Encapsulation protects the smart-card balance.
- Integer paise avoids money-related rounding errors.
- A central fare table makes fare changes easier.
- enum class prevents invalid zone names from being used accidentally.
- Separate functions make the program easier to test and maintain.
- The design supports future improvements such as additional zones and monthly passes.

3. Limitations

- A compiled fare table requires recompilation when fares change.
- Passenger concession rates must be confirmed before implementation.
- Adding more zones requires updating the zone declaration and fare table.
- Monthly-pass functionality needs separate validation and management.

4. Alternative Configuration-File Design

Instead of storing fares directly in the program, the system can read the fare values from a configuration file during startup.

Advantages:

- Fare changes can be made without changing the main program code.

- The fare table can be updated easily.
- The same program can support different fare configurations.

Limitations:

- The file must be validated.
- The program must handle missing or incorrect files.
- Secure access and configuration versioning are required

5. Code:

```
#include <iostream>

using namespace std;


constexpr long long MIN_BALANCE = 5000;


enum class Zone {

    Inner,

    Middle,

    Outer

};


class FareTable {

public:

    static long long getFare(Zone z, bool peak) {

        long long fare = 0;
```

```
        switch (z) {
            case Zone::Inner:
                fare = 1000;
                break;

            case Zone::Middle:
                fare = 2000;
                break;

            case Zone::Outer:
                fare = 3000;
                break;

            default:
                return -1;
        }

        if (peak) {
            fare += fare * 20 / 100;
        }

        return fare;
    }
};
```

```
class SmartCard {
private:
    long long balance;

public:
    SmartCard(long long b) {
        balance = b;
    }

    bool canTravel(long long fare) const {
        return fare >= 0 &&
               balance - fare >= MIN_BALANCE;
    }

    void deduct(long long fare) {
        if (canTravel(fare)) {
            balance -= fare;
            cout << "Journey Approved\n";
        } else {
            cout << "Journey Denied\n";
        }
    }
```

```
    long long getBalance() const {
        return balance;
    }
};

int main() {
    SmartCard card(10000);

    long long fare = FareTable::getFare(
        Zone::Outer, true
    );

    cout << "Calculated Fare: Rs "
         << fare / 100 << '\n';

    card.deduct(fare);

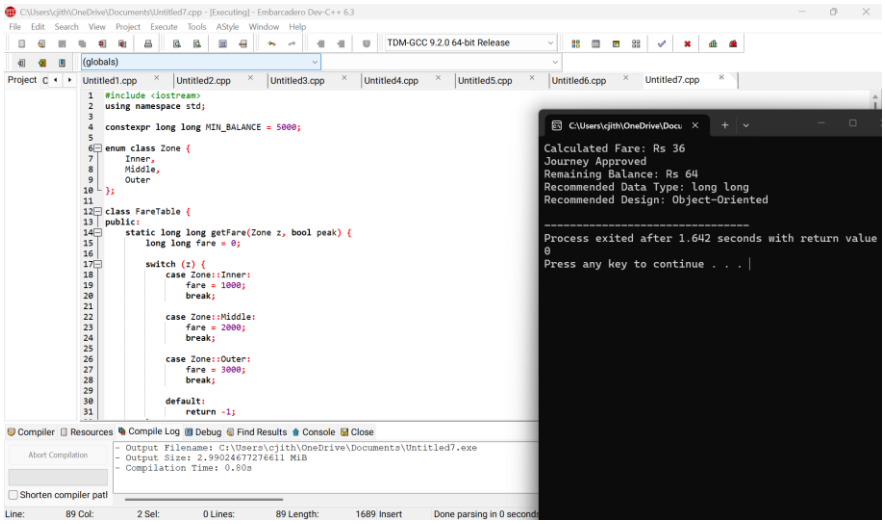
    cout << "Remaining Balance: Rs "
         << card.getBalance() / 100 << '\n';

    cout << "Recommended Data Type: long long\n";
    cout << "Recommended Design: Object-Oriented\n";

    return 0;
```

}

6.Output:



5. Complexity Analysis

Fare calculation takes O(1) time. A configuration file containing z zone values takes O(z) time to load and O(z) memory to store.

The recommended design is suitable for a large metro system because it provides simple fare calculation, safe balance handling and easier maintenance.

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

A. Impact Analysis

The metro system is updated with three new features:

1. Airport Zone: A new zone is added with a base fare of Rs. 60.
2. Weekend Discount: A 10% discount is applied during weekends. It must not be combined with the peak-hour surcharge.
3. Monthly Pass: Passengers can select two zones and travel unlimited times within those zones during the pass validity period.

The following changes are required:

- Add Airport to the Zone enumeration.
- Update the fare table to include four zones.
- Update switch statements and zone validation.
- Add weekend information to journey input.
- Add a separate Pass class to manage monthly-pass details.
- Check whether the selected journey is covered by the monthly pass.

B. Evaluation of the Original Design

The original switch statement can be updated by adding another case for Airport. However, repeated switch statements become difficult to maintain when more zones are added.

A central fare table is easier to extend.

The SmartCard class can continue managing the stored balance. Monthly-pass details should be handled separately using a Pass class.

C. Revised Fare Calculation Examples

Journey	**Fare**
Airport weekday off-peak	Rs. 60
Airport weekday peak	Rs. 72
Airport weekend	Rs. 54
Airport with valid monthly pass	Rs. 0

The weekend discount replaces the peak surcharge. Both benefits are not combined.

D. Revised C++ Program

```
#include <iostream>

#include <array>

using namespace std;

enum class Zone {
```

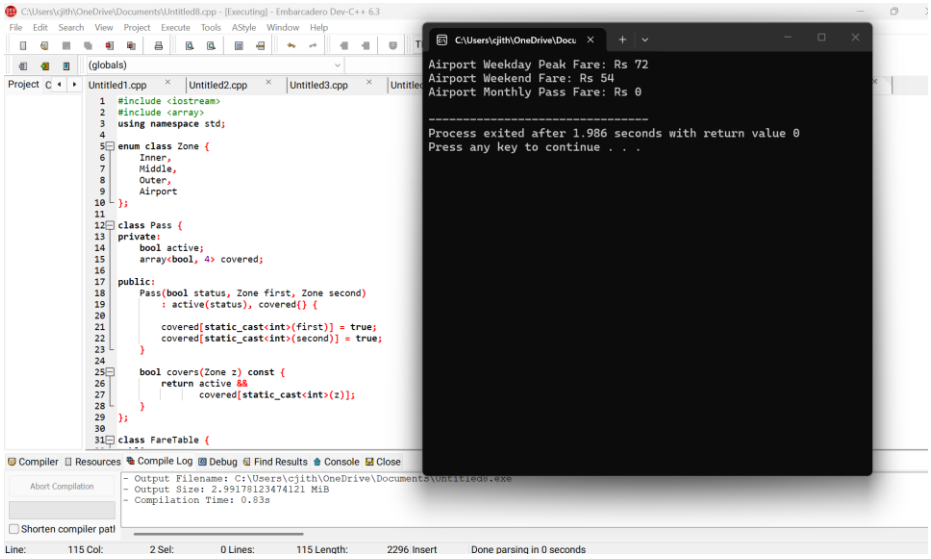
```
    Inner,
    Middle,
    Outer,
    Airport
};
class Pass {
private:
    bool active;
    array<bool, 4> covered;
public:
    Pass(bool isActive, Zone first, Zone second)
        : active(isActive), covered{} {
        covered[static_cast<int>(first)] = true;
        covered[static_cast<int>(second)] = true;
    }
    bool covers(Zone z) const {
        return active && covered[static_cast<int>(z)];
    }
};
long long baseFare(Zone z) {
    constexpr array<long long, 4> fares = {
        1000, 2000, 3000, 6000
    };
    return fares[static_cast<int>(z)];
```

```
}
long long calculateFare(
    Zone z,
    int hour,
    bool weekend,
    const Pass& pass
) {
    if (hour < 0 || hour > 23) {
        return -1;
    }
    if (pass.covers(z)) {
        return 0;
    }
    long long fare = baseFare(z);
    bool peak = (hour >= 8 && hour < 10) ||
                (hour >= 17 && hour < 20);
    if (weekend) {
        fare -= fare * 10 / 100;
    }
    else if (peak) {
        fare += fare * 20 / 100;
    }
    return fare;
}
```

```
int main() {
    Pass monthlyPass(
        true,
        Zone::Inner,
        Zone::Airport
    );
    cout << "Airport weekday peak: "
         << calculateFare(
                Zone::Airport,
                18,
                false,
                Pass(false, Zone::Inner, Zone::Outer)
            )
         << " paise\n";
    cout << "Airport weekend: "
         << calculateFare(
                Zone::Airport,
                18,
                true,
                Pass(false, Zone::Inner, Zone::Outer)
            )
         << " paise\n";
    cout << "Airport with pass: "
         << calculateFare(
```

```
            Zone::Airport,
            12,
            false,
            monthlyPass
        )
    << " paise\n";
    return 0;
}
```

E. Expected Output



F. Explanation of the Revised Design

- The Airport zone is added to the existing four-zone enumeration.
- The fare table stores all four base fares.
- A 20% surcharge is applied during weekday peak hours.
- A 10% discount is applied on weekends instead of the peak surcharge.
- The Pass class stores the two selected zones.
- If a valid pass covers the journey zone, the fare is zero.
- If the journey is not covered by the pass, the normal fare calculation is performed.

G. Complexity Analysis

Fare lookup and monthly-pass checking take O(1) time.

The fare table and pass coverage array use O(1) space because the current system contains four zones.